

INDEX-COVERAGE

Code-Intelligence Index Coverage — Verified Report

Repository: <ext-volume-host>/Projects/vasic **Verification run:** 2026-08-26, 20:37Z – 20:46Z (local 22:37 – 22:46, UTC+2) **Mode:** read-only. No index was built, rebuilt, purged, synced or deleted. Every number below comes from a command reproduced verbatim in this document.

Live-data warning. A lumen index run on this repository was **in flight for the whole verification window** (PID 1777428, lumen index <ext-volume-host>/Projects/vasic, elapsed 11m47s at the last sample). All Lumen numbers are a **moving snapshot**, and are labelled with the sample timestamp. CodeGraph numbers are static — its index reports `index_state = complete`.

0. TL;DR

Indexer	Umbrella proper	Submodules	Verdict
CodeGraph v1.5.0	150 files	391 files across 6 of 7 submodules	Coverage is complete for the languages CodeGraph supports . Its file count matches the on-disk eligible count <i>exactly</i> (541 = 541). The gap is not

Indexer	Umbrella proper	Submodules	Verdict
Lumen v0.0.41	496 files (13% of target)	0 files — none, in any submodule	“missing submodules”, it is a narrow language set (no Markdown / HTML / shell / TOML / JSON). Submodules are in Lumen’s walk scope (proved by cross-project control), but the in-flight run has not reached a single submodule directory yet. ~1,840 umbrella files still sit ahead of the first submodule in walk order.

Single biggest gap: every one of the 7 direct submodules and all 7 nested constitution submodules has **zero** semantic (Lumen) coverage — 1,449 indexable files, 0 indexed.

1. What is actually in the repository

7 direct submodules, from .gitmodules:

```
git -C <ext-volume-host>/Projects/vasic submodule status
023abbfdfe12a604144cf420d1ec3d9efaa6e89c ai_interviewing (heads/main)
16e4e76d57ab61b5f3b46fae3372b2e6d6cc73e3 design-toolkit (v0.2.2-4-g16e4e76)
```

```
8385025500ce0595b1fad3ff3fb4c58e93b88504 milosvasic.ru (v1.8.0-5-
g8385025)
1f9f52042a81c9a2d0e0f2f42e3ca9fbf4d7fbfe monetization (heads/
main)
448981ae3498229c734dc60719f4b19f01d7a75f submodules/constitution
(v1.0.0-51-g448981a)
c20ac6c1ba069cc9a72dacb8044b7b193d3dde81 submodules/superspec
(v1.0.1-7-gc20ac6c)
5a4c3bba8689e9d2c35887b77ee0a6c3da6248b6 vasic.digital (v1.8.0-4-
g5a4c3bb)
```

All 7 are **checked out and populated** (no empty submodule directories). The 7 nested submodules under submodules/constitution/submodules/ are likewise populated:

```
git -C submodules/constitution submodule status
```

```
bcc7f3d... submodules/anti_bluff (heads/main)
d88a007... submodules/continuum (heads/main)
16e4e76... submodules/design-toolkit (v0.2.2-4-g16e4e76)
9510e01... submodules/docs_chain (helixcode-v1.1.0-9-g9510e01)
6adcfb2... submodules/helix_perf_cache (heads/main)
78b77bb... submodules/session_orchestrator (heads/main)
c0591b5... submodules/token_optimizer (heads/main)
```

2. CodeGraph coverage

2.1 Index metadata

```
DB="file:<ext-volume-host>/Projects/vasic/.codegraph/codegraph.db?
mode=ro"
```

```
sqlite3 "$DB" "SELECT * FROM project_metadata;"
```

```
index_state|complete|1787769288918
indexed_with_version|1.5.0|1787769288918
indexed_with_extraction_version|24|1787769288918
index_files_discovered|541|1787769288918
index_files_accounted|541|1787769288918
```

```
sqlite3 "$DB"
"SELECT 'nodes',COUNT(*) FROM nodes UNION ALL SELECT
'edges',COUNT(*) FROM edges UNION ALL SELECT
'unresolved_refs',COUNT(*) FROM unresolved_refs;"
```

```
nodes|10286
edges|39793
unresolved_refs|33185
```

The index is **complete and self-consistent**: discovered 541, accounted 541, files table holds 541 rows.

2.2 Per-submodule coverage table

```
-- sqlite3 -header -column "$DB"
WITH s AS (
  SELECT path, language, indexed_at,
    CASE
      WHEN path LIKE 'ai_interviewing/%' THEN
        'ai_interviewing'
      WHEN path LIKE 'design-toolkit/%' THEN 'design-
        toolkit'
      WHEN path LIKE 'milosvasic.ru/%' THEN 'milosvasic.ru'
      WHEN path LIKE 'monetization/%' THEN 'monetization'
      WHEN path LIKE 'submodules/constitution/%' THEN 'submodules/
        constitution'
      WHEN path LIKE 'submodules/superspec/%' THEN 'submodules/
        superspec'
      WHEN path LIKE 'vasic.digital/%' THEN 'vasic.digital'
      ELSE '(umbrella proper)' END AS scope
    FROM files)
SELECT scope, COUNT(*) n, GROUP_CONCAT(DISTINCT language) langs,
  datetime(MAX(indexed_at)/1000, 'unixepoch') newest
FROM s GROUP BY scope ORDER BY n DESC;
```

Scope	Indexed files	Languages present	Newest indexed_at (UTC)	Tracked files on disk	Coverage of tracked
submodules/constitution (incl. nested)	331	yaml, python, go, javascript	2026-08-26 18:34:43	1,185 (+ nested)	~28%
(umbrella proper)	150	yaml, python, javascript, xml, go	2026-08-26 18:34:42	5,677	2.6%
ai_interviewing	23	go, typescript	2026-08-26 18:34:42	911	2.5%
design-toolkit	19	javascript	2026-08-26 18:34:43	90	21%
milosvasic.ru	10	yaml, xml, javascript	2026-08-26 18:34:43	1,017	1.0%
submodules/superspec	5	yaml, python	2026-08-26 18:34:43	44	11%
vasic.digital	3			1,130	0.3%

Scope	Indexed files	Languages present	Newest indexed_at (UTC)	Tracked files on disk	Coverage of tracked
		javascript, xml	2026-08-26 18:34:43		
monetization	0	— (ZERO INDEXED FILES)	—	8	0%

Nested constitution submodules (166 of the 331 above):

```
WITH x AS (SELECT substr(path,36) AS rest, language, indexed_at
FROM files WHERE path LIKE 'submodules/constitution/
submodules/%')
SELECT substr(rest,1,instr(rest,'/')->1) AS nested, COUNT(*) n,
GROUP_CONCAT(DISTINCT language) langs,
datetime(MAX(indexed_at)/1000,'unixepoch') newest
FROM x GROUP BY nested ORDER BY n DESC;
```

Nested submodule	Indexed files	Languages	Newest indexed_at (UTC)	Tracked files
token_optimizer	58	yaml, go	2026-08-26 18:34:43	72
docs_chain	41	yaml, go	2026-08-26 18:34:43	87
continuum	21	go, yaml	2026-08-26 18:34:43	31
design-toolkit	19	javascript	2026-08-26 18:34:43	90
session_orchestrator	13	go, yaml	2026-08-26 18:34:43	20
helix_perf_cache	13	go, yaml	2026-08-26 18:34:43	24
anti_bluff	1	yaml	2026-08-26 18:34:43	27

submodules/constitution proper (excluding nested): **165** files.

2.3 Overall language distribution

```
sqlite3 "$DB" "SELECT language, COUNT(*) FROM files GROUP BY
          language ORDER BY 2 DESC;"
```

```
go|287
javascript|132
python|47
yaml|44
xml|17
typescript|14
```

No markdown, no html, no shell, no json, no toml, no css.

3. WHY the low / zero CodeGraph numbers — proven, not guessed

The empirical CodeGraph extension set derived from the language column is .go .js .mjs .cjs .jsx .py .yaml .xml .ts .tsx. Counting those on disk:

```
cd <ext-volume-host>/Projects/vasic
CEXT='go|js|mjs|cjs|jsx|py|yaml|xml|ts|tsx'
SKIP='-name .git -prune -o -name node_modules -prune -o
      -name .venv -prune -o -name __pycache__ -prune
      -o -name dist -prune -o -name build -prune -o -name target
      -prune -o -name out -prune
      -o -name vendor -prune -o -name .ptmp -prune'
c(){ find "$1" $SKIP -o -type f -print 2>/dev/null | grep -Ei "\.
      ($CEXT)$" | wc -l; }
c . # -> 541
for p in ai_interviewing design-toolkit milosvasic.ru
      monetization \
      submodules/constitution submodules/superspec
      vasic.digital; do echo "$p: $(c ./$p)"; done
```

Scope	Eligible on disk	In CodeGraph DB	Match?
whole tree	541	541	exact
ai_interviewing	23	23	
design-toolkit	19	19	
milosvasic.ru	10	10	
monetization	0	0	
	331	331	

Scope	Eligible on disk	In CodeGraph DB	Match?
submodules/ constitution			
submodules/superspec	5	5	
vasic.digital	3	3	

Conclusion: CodeGraph is not missing any submodule. Every eligible file on disk is in the index. The apparent gaps are entirely explained by file *type*, proved per submodule:

```
git -C <submodule> ls-files | sed 's/.*\./' | sort | uniq -c |
    sort -rn | head -20
```

Submodule	Dominant tracked file types	Why CodeGraph coverage is low/zero
monetization	4 sh, 2 md, 1 txt, 1 gitignore	Docs+shell-only repo. Contains literally zero files in any CodeGraph-supported language. Zero is <i>correct</i> , not a bug.
vasic.digital	795 html, 273 md, 20 woff2, 15 pdf, 6 txt, 5 css, 3 json, 3 js	A static website. Only the 3 .js + 1 .xml are eligible; CodeGraph holds 3.
milosvasic.ru	707 html, 193 md, 52 pdf, 12 woff2, 7 css, 6 js, 4 sh, 4 json, 3 xml	Same — static website; 6 js + 3 xml + yaml ⇒ 10.
ai_interviewing	159 png, 159 mmd, 159 md, 101 svg, 81 json, 69 pdf, 69 html, 68 docx, 14 ts, 14 sh, 9 go	Diagram/report-heavy. Only 14 ts + 9 go = 23 eligible.
submodules/ constitution	194 sh, 179 md, 172 html, 171 pdf, 171 docx, 125 go, 36 toml, 29 txt, 23 yaml, 16 py	Governance repo: shell + Markdown + generated PDF/DOCX dominate; only Go/YAML/Python are eligible.

Submodule	Dominant tracked file types	Why CodeGraph coverage is low/zero
submodules/superspec	31 md, 3 yml, 2 sh, 2 py, 2 png	Spec/docs repo; 3 yml + 2 py = 5 eligible.
design-toolkit	43 md, 19 mjs, 7 txt, 7 json, 5 css, 4 svg	19 .mjs eligible ⇒ 19 indexed.
.../anti_bluff	13 sh, 8 txt, 2 sql, 1 yaml, 1 md	Shell-only tooling; 1 yaml eligible ⇒ 1 indexed.

No submodule is “genuinely missed” by CodeGraph, and no submodule is excluded by a skip rule (vendor/, dist/, node_modules/). The single failure mode is that CodeGraph does not parse Markdown, HTML, shell or TOML — which is precisely the content that dominates monetization, vasic.digital, milosvasic.ru and constitution.

4. Lumen coverage

4.1 Where the index lives, and how it was found

Lumen keeps **one index directory per project root**, hashed, under `~/.local/share/lumen/<16-hex>/index.db`. 763 such directories exist on this machine. The one for this repository was located by reading `project_meta` out of every one of them:

```
cd <home>/~/.local/share/lumen
for d in */; do d="${d%/}"; [ -f "$d/index.db" ] || continue
echo "$d|$(sqlite3 "file:$PWD/$d/index.db?mode=ro" \
  "SELECT (SELECT value FROM project_meta WHERE
    key='project_path')||'||' \
    COALESCE((SELECT value FROM project_meta WHERE
    key='total_files'),'?')||'||' \
    COALESCE((SELECT value FROM project_meta WHERE
    key='last_indexed_at'),'?');")"
done | grep -i 'Projects/vasic'
```

```
21bf1507a8925bcf|<ext-volume-host>/Projects/vasic|3815|
2026-08-26T20:34:17Z
```

Exactly one Lumen index exists for this project. A `grep` of all 763 project paths for any submodule name returns **nothing** — no submodule has its own Lumen index:

```
grep -Ei 'ai_interviewing|design-toolkit|milosvasic\.ru|
monetization|constitution|superspec|vasic\.digital|vasic-
digital' lumen_projects.txt
# (no output)
```

4.2 What the index contains right now

```
L="file:<home>/.local/share/lumen/21bf1507a8925bcf/index.db?
mode=ro"
sqlite3 "$L" "SELECT * FROM project_meta;"

vec_dimensions|768
root_hash|
ba64d624e6a005a24c94642f37cbdcdba9b5b36f1ab2f575cbb1414e28894268
embedding_model|ordis/jina-embeddings-v2-base-code
project_path|<ext-volume-host>/Projects/vasic
last_indexed_at|2026-08-26T20:34:17Z
total_files|3815

SELECT CASE WHEN instr(path, '/')>0 THEN
    substr(path,1,instr(path, '/')-1) ELSE '(root file)' END
AS top,
COUNT(*) FROM files GROUP BY top ORDER BY 2 DESC;
```

Sample at 2026-08-26T20:39:20Z — 496 files, 29,221 chunks:

Top-level dir	Files
_analysis	73
_content	69
.specify	67
_content_fa / _es / _de / _be / _ar	38 each
_content_fr	36
.test-evidence	13
.opencode / .kimi-code / .claude	10 each
.github	1
(root file)	1
any submodule directory	0

Explicit per-submodule check:

```
for p in ai_interviewing design-toolkit milosvasic.ru
monetization \
submodules/constitution submodules/superspec
vasic.digital; do
```

```

    echo "$p = $(sqlite3 "$L"
        "SELECT COUNT(*) FROM files WHERE path LIKE '$p/%';")"
done

ai_interviewing = 0
design-toolkit = 0
milosvasic.ru = 0
monetization = 0
submodules/constitution = 0
submodules/superspec = 0
vasic.digital = 0

```

4.3 Does Lumen index submodules at all? — control experiment

This is the decisive question, and it is answered with a **control**, not an assumption. Two other repositories on this machine have completed Lumen indexes and use git submodules:

```

grep -E '^s*path' <ext-volume-host>/Projects/lava/.gitmodules
# 20 submodules
grep -E '^s*path' <ext-volume-host>/Projects/boba/.gitmodules
# 6 submodules

sqlite3 "file:<home>/.local/share/lumen/feaae19c7a38712e/index.db?
mode=ro" \
    "SELECT CASE WHEN instr(path,'/')>0 THEN
        substr(path,1,instr(path,'/')-1) ELSE '(root)' END t,
        COUNT(*) FROM files GROUP BY t ORDER BY 2 DESC LIMIT 5;"

```

lava (7,692 files indexed):

```

submodules|4292
lava-api-go|1845
constitution|586      <- a git submodule
.lava-ci-evidence|338
docs|283

```

boba (38,406 discovered):

```

submodules|12828
constitution|617      <- a git submodule
docs|458

```

Proof: Lumen walks *into* git submodule directories and indexes them as part of the umbrella project. Submodules do **not** need their own Lumen index.

4.4 Second proof — the target file count includes submodules

Lumen's supported extension set, derived empirically from the two *completed* control indexes, is:

.c .h .go .ts .tsx .js .jsx .mjs .rs .rb .cc .cs .cpp .cxx .hpp .p
hp .py .java .dart .md .mdx .yaml .yml .json (corroborated by literal
blobs in the binary:

.ts.js.py.rs.rb.cc.cs.md, .tsx.jsx.mjs.cpp.cxx.hpp.php.mdx.yml, .java.dart.yaml.json

Notably **absent**: .sh, .html, .css, .kt, .toml, .sql.

Counting that set across this repository:

```
LEXT='c|h|go|ts|js|py|rs|rb|cc|cs|md|tsx|jsx|mjs|cpp|cxx|hpp|php|  
mdx|yml|java|dart|yaml|json'  
SKIP='-name .git -prune -o -name node_modules -prune -o  
-name .venv -prune -o -name venv -prune  
-o -name __pycache__ -prune -o -name dist -prune -o -name  
build -prune -o -name target -prune  
-o -name out -prune -o -name .next -prune -o -name .nuxt  
-prune -o -name .eggs -prune  
-o -name .idea -prune -o -name vendor -prune -o -name  
coverage -prune -o -name .ptmp -prune'  
c(){ find "$1" $SKIP -o -type f -print 2>/dev/null | grep -Ei "\.  
($LEXT)$" | wc -l; }  
c . # 3863
```

Measure	Files
Whole tree, Lumen-eligible	3,863
Lumen's own total_files target	3,815 (98.8% of 3,863)
Umbrella proper only, Lumen-eligible	2,414

3,815 matches the **whole-tree** figure, **not** the umbrella-only figure (2,414). The 48-file delta is .gitignore filtering (Lumen links github.com/sabhiram/go-gitignore and has merkle.NewIgnoreTree / merkle.globalGitignorePath symbols). **Lumen has already discovered and planned to index the submodule files — it simply has not reached them yet.**

4.5 Where the in-flight run currently is

```
ps -o pid,etime,time,%cpu -p 1777428  
# PID ELAPSED TIME %CPU  
# 1777428 11:47 00:00:09 1.4  
sqlite3 "$L" "SELECT path FROM files ORDER BY rowid DESC LIMIT 3;"  
# _content_hi/products/HelixLLM.md
```

```
# _content_hi/products/HelixGitpx.md
# _content_hi/products/HelixConstitution.md
```

The walk is alphabetical: dot-directories → `_`-directories → lowercase names. It is currently inside `_content_hi`. The first submodule in walk order is `ai_interviewing`.

```
for d in _content_hi _content_ja _content_kk _content_ko
        _content_ru _content_sr \
        _content_tr _content_zh _tests _tools; do echo "$d: $
(c ./ $d)"; done

_content_hi: 38    _content_ja: 38    _content_kk: 38    _content_ko:
38
_content_ru: 38    _content_sr: 38    _content_tr: 38    _content_zh:
38
_tests: 1473      _tools: 63
-> ~1,840 files still ahead of the first submodule
```

~1,840 files (dominated by `_tests/`, 1,473 evidence JSON/MD files) must be embedded before Lumen touches `ai_interviewing`.

4.6 Measured throughput (this machine, right now)

Sample (UTC)	files	chunks
20:37:56	480	28,962
20:39:20	496	29,221
20:42:15	496	29,221
20:42:58	496	29,221
20:43:44	496	29,221
20:44:29	496	29,221
20:46:11	496	29,221
20:48:41	508	—

Net observed: **28 files in 10m45s ≈ 2.6 files/min** — squarely inside the stated 1–3 files/min contended band. `cat /proc/loadavg` reported 31.40 27.96 24.70 (a concurrent Android emulator matrix is saturating the box); the ollama runner for `ordis/jina-embeddings-v2-base-code` has burned 1m56s CPU in 11m wall.

5. The honest gap list (ranked)

#	Gap	Indexer	Evidence	Files affected	Fix command (NOT RUN)
1	All 7 direct + 7 nested submodules have ZERO semantic coverage	Lumen	SELECT COUNT(*) FROM files WHERE path LIKE '<sub>/%' returns 0 for every submodule (\$4.2)	1,449	lumen index <ext- volume- host>/ Projects/ vasic — already running (PID 1777428). No new command needed; it needs <i>time</i> . Unfixable & not a defect.
2	monetization has zero CodeGraph coverage	CodeGraph	0 eligible files on disk = 0 in DB (\$3)	8 tracked / 2 Lumen-eligible	Repo is 4 sh + 2 md + 1 txt. Lumen will cover the 2 .md. No CodeGraph fix exists (it has no HTML/ Markdown parser). Covered by Lumen once it reaches the
3	vasic.digital — 3 of 1,130 tracked files indexed by CodeGraph	CodeGraph	795 html, 273 md vs. CodeGraph's language set (\$3)	1,127	

#	Gap	Indexer	Evidence	Files affected	Fix command (NOT RUN)
					directory: 278 eligible files.
4	milosvasic.ru — 10 of 1,017 tracked files indexed by CodeGraph	CodeGraph	707 html, 193 md (§3)	1,007	Same as above. Lumen will cover 207 files.
5	ai_interviewing — 23 of 911 tracked files indexed by CodeGraph	CodeGraph	159 png/mmd/md, 101 svg, 81 json, 69 pdf/html, 68 docx (§3)	888	Same. Lumen will cover 264 files.
6	submodules/constitution — 331 of 1,185+ tracked	CodeGraph	194 sh, 179 md, 172 html, 171 pdf, 171 docx, 36 toml (§3)	~850	Same. Lumen will cover 593 files. .sh is covered by neither indexer.
7	.../anti_bluff — 1 of 27 tracked	CodeGraph	13 sh, 8 txt, 2 sql, 1 yaml, 1 md (§3)	26	Same. Lumen covers only the 1 .md + 1 .yaml.
8	Shell scripts are invisible to both indexers	Both	CodeGraph language has no shell; Lumen ext set has no .sh (§2.3, §4.4)	194 in constitution, 13 in anti_bluff, 14 in ai_interviewing, 4 in monetization, ...	No supported fix in either tool at these versions.
9	763 stale Lumen project dirs on disk (98	Lumen	ls ~/.local/share/lumen \	—	lumen purge ... — not run,

#	Gap	Indexer	Evidence	Files affected	Fix command (NOT RUN)
	MB for this project alone)		<code>wc -l</code> → 764 (incl. <code>debug.log</code>)		and out of scope of this task.

Questions that cannot be answered without a mutating command

- “Will the current Lumen run definitely complete all 3,815 files?” — cannot be confirmed by observation; it depends on the run not being killed. Confirming would require waiting for exit, or `lumen index ...` again (mutating).
- “Is CodeGraph’s index stale relative to today’s working tree?” — `codegraph status` answers this but opens the DB **for writing** (migrations / FTS heal), so it was not run. The proxy evidence (541 on disk == 541 in DB, `index_state=complete`) says it is current.
- “Would a CodeGraph re-index pick up more?” — `codegraph sync <path>` would answer it, and is mutating. Based on §3 the answer is almost certainly *no* — its file count already matches disk exactly.

6. Cost estimate for the outstanding Lumen work

Baseline: **1–3 files/min** contended (measured here: **1.9 files/min**); ~20 s for a 1–2 file fixture on an idle backend.

Work item	Files	@1 file/ min	@1.9 files/ min (measured)	@3 files/ min	On an idle backend (~30 files/ min)
Finish umbrella proper (reach first submodule)	~1,840	30.7 h	16.1 h	10.2 h	~1.0 h

Work item	Files	@1 file/ min	@1.9 files/ min (measured)	@3 files/ min	On an idle backend (~30 files/ min)
ai_interviewing	264	4.4 h	2.3 h	1.5 h	~9 min
design-toolkit	69	1.2 h	36 min	23 min	~2 min
milosvasic.ru	207	3.5 h	1.8 h	1.2 h	~7 min
monetization	2	2 min	1 min	<1 min	~20 s
submodules/ constitution (incl. 7 nested)	593	9.9 h	5.2 h	3.3 h	~20 min
submodules/ superspec	36	36 min	19 min	12 min	~1 min
vasic.digital	278	4.6 h	2.4 h	1.5 h	~9 min
All submodules	1,449	24.2 h	12.7 h	8.1 h	~48 min
Everything still outstanding (3,815 – 496)	3,319	55.3 h	29.1 h	18.4 h	~1.8 h

Practical read: the dominant cost is not the submodules (1,449 files, 8–24 h) — it is `_tests/` (1,473 evidence files) sitting *ahead* of them in walk order. The single highest-leverage, zero-risk action is to **free the embedding backend** (the concurrent Android emulator matrix is holding load average at ~31); at an idle-backend rate the entire remaining job is under two hours instead of over a day.

Cost for CodeGraph: **zero**. It is complete for everything it is able to parse.

7. Reproducibility notes

- CodeGraph DB was opened **read-only** throughout:
`file:.../.codegraph/codegraph.db?mode=ro.codegraph init / index /
uninit / unlock / sync / status` were **never** invoked.

- Lumen DBs were opened **read-only**: `file:.../index.db?mode=ro`.
`lumen index / lumen purge / lumen search` were **never** invoked. Only
`lumen --help`, `lumen index --help`, `lumen search --help`, `lumen`
`version` (all read-only) were run.
- Binary-literal inspection used `strings` on `~/.claude-shared/`
`plugins/cache/claude-plugins-official/lumen/0.0.41/bin/lumen-`
`linux-amd64`.
- The only file written by this verification is this document.